МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационных технологий»

Лабораторная работа № 4
по дисциплине «Введение в информационные технологии»
по теме: «**ОРГАНИЗАЦИЯ PYTHON-ПРОЕКТА**»

Выполнил студент группы:
БВТ2601
Гутаев Эльдар Назирович
Проверил:

Москва 2026

Оглавление

Введение……3

Выполнение 4

Вывод..... 26

Введение

Цель и задачи работы:

Целью данной лабораторной работы является: изучить принципы организации Python-проекта, научиться разделять программу на модули и пакеты, использовать способы импорта, работать с файловой системой, обрабатывать исключения, применять логирование, использовать конфигурационные файлы, создавать виртуальные окружения и управлять зависимостями проекта.

Задачи: В ходе выполнения лабораторной работы необходимо научиться организовывать структуру проекта, распределять программный код между несколькими файлами, корректно обрабатывать ошибки, сохранять диагностическую информацию в журнал, выносить изменяемые параметры в конфигурационный файл и создавать воспроизводимое окружение проекта.

Выполнение

Задание 1. На рисунке 1: создан проект measurement_project, открытый в Visual Studio Code. В корне проекта находятся main.py, config.json и .gitignore; создан пакет utils с файлами __init__.py, reader.py и statistics.py; созданы каталоги data и logs. Среда VS Code не позволяет создать пустой каталог без файлов в нужном виде, поэтому в data и logs добавлены пустые файлы-заглушки .gitkeep. Они нужны только для того, чтобы каталоги присутствовали в проекте и в Git; на работу программы они не влияют.

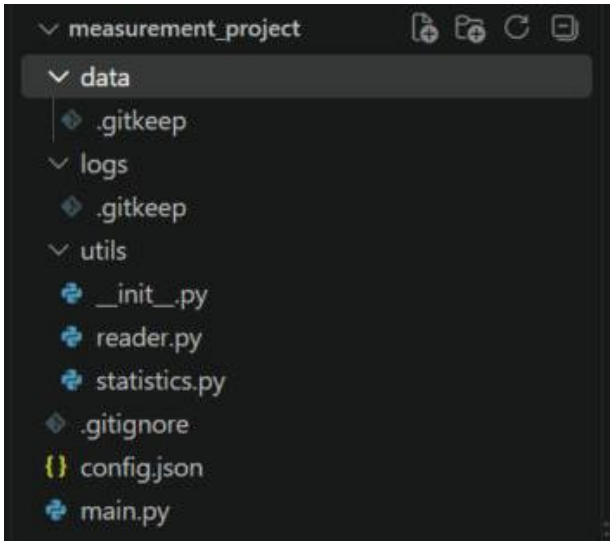


Рисунок 1. Структура проекта после создания

Задание 2. На рисунке 2: в модуле utils/statistics.py созданы функции calculate_average(), find_min() и find_max(). В main.py они подключены конструкцией from utils.statistics import ..., для тестового списка [10, 20, 30, 40, 50] выведены среднее 30.0, минимум 10 и максимум 50.

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)


def find_min(numbers):
    return min(numbers)


def find_max(numbers):
    return max(numbers)
```

```
from utils.statistics import calculate_average, find_min,
find_max

numbers = [10, 20, 30, 40, 50]
print("Среднее:", calculate_average(numbers))
print("Минимум:", find_min(numbers))
print("Максимум:", find_max(numbers))
```

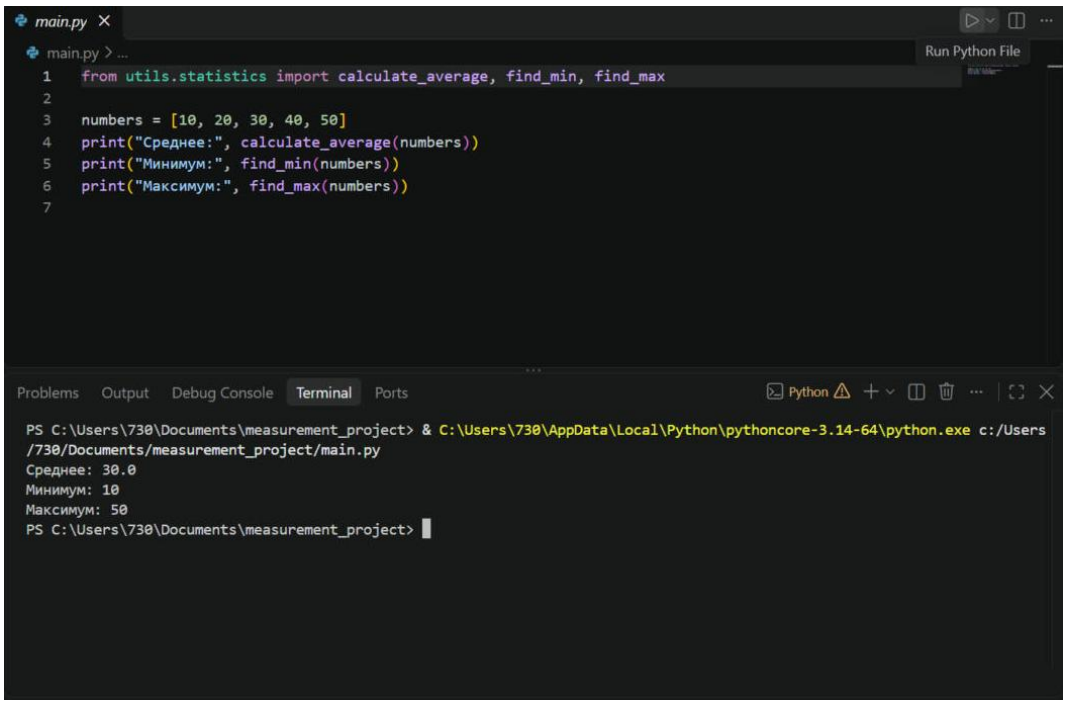


Рисунок 2. Модуль statistics и его использование в main.py

Задание 3. На рисунке 3: код тестовой обработки перенесён в функцию main(), которая вызывается под условием if __name__ == "__main__". При прямом запуске main.py значение __name__ равно "__main__", поэтому функция выполняется и выводит результат. На рисунке 3.1: временный файл test_import.py импортирует main.py как модуль. Вывод показывает, что имя модуля — main, а результаты вычислений не выводятся: при импорте __name__ равно имени модуля, условие ложно и main() автоматически не

запускается. Это позволяет использовать функции файла в других модулях без запуска всей программы.

```
from utils.statistics import calculate_average, find_min,
find_max


def main():
    numbers = [10, 20, 30, 40, 50]
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))


if __name__ == "__main__":
    main()
```

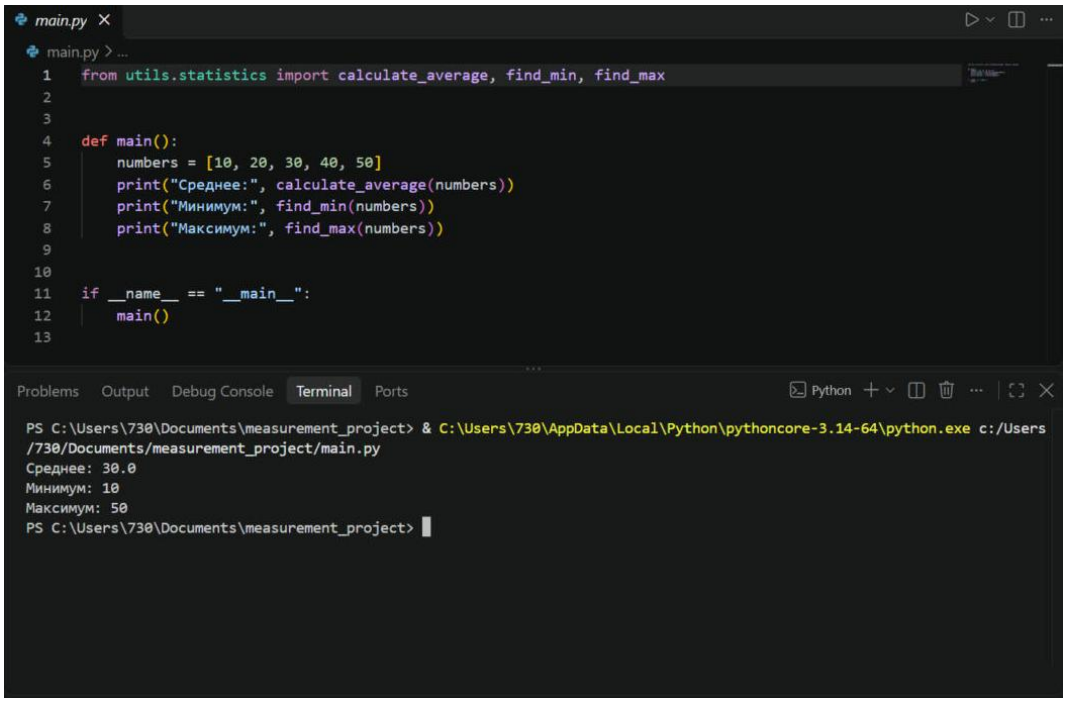


Рисунок 3. Запуск main.py с точкой входа

```
import main

print("Модуль main импортирован, имя модуля:", main.__name__)
print("Функция main() автоматически не запускалась")
```

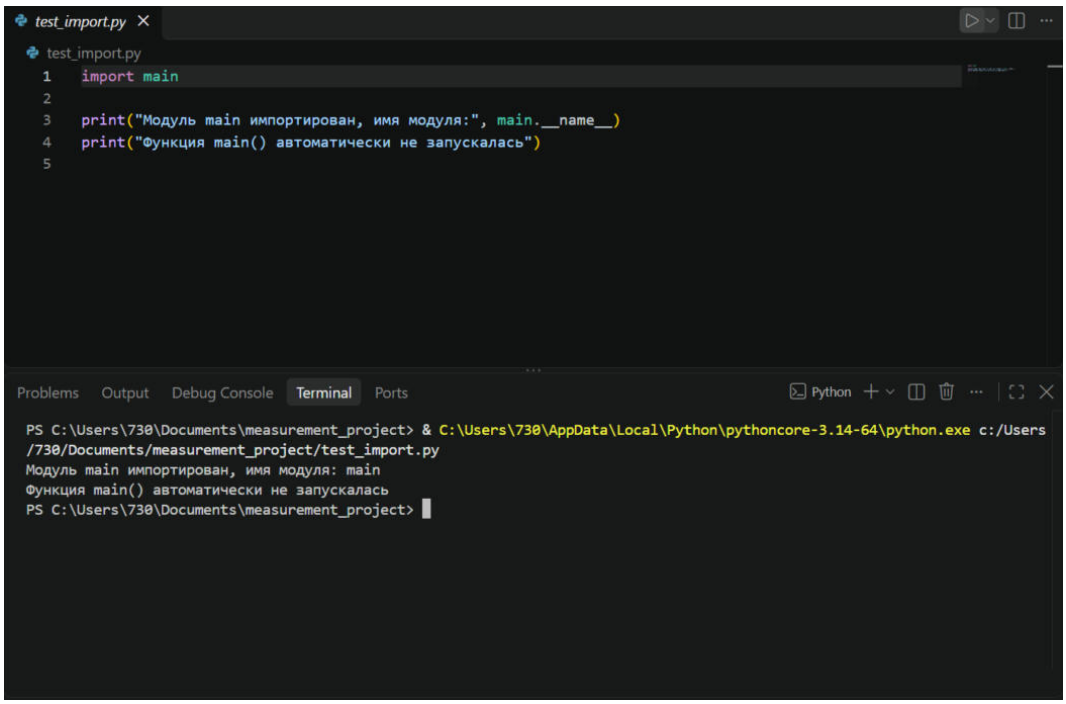


Рисунок 3.1. Импорт main.py как модуля

Задание 4. На рисунке 4: создан файл data/measurements.txt с 12 числовыми значениями, по одному в строке. В utils/reader.py реализована функция read_numbers(file_path): она открывает файл, читает строки, пропускает пустые, преобразует каждую строку в float и возвращает список. На рисунке 4.1: main.py использует read_numbers() вместо тестового списка и выводит количество значений (12), среднее (22.54), минимум (20.8) и максимум (24.2).

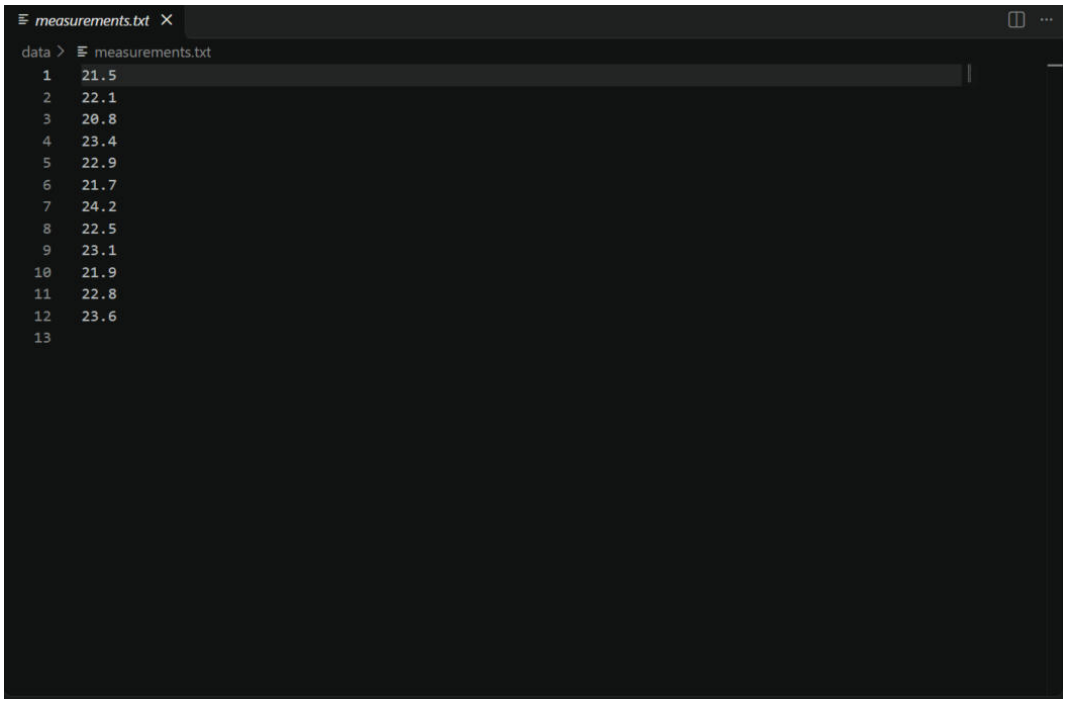


Рисунок 4. Файл measurements.txt

```
def read_numbers(file_path):
    numbers = []
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            line = line.strip()
            if line:
                numbers.append(float(line))
    return numbers

from utils.reader import read_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    numbers = read_numbers("data/measurements.txt")
    print("Прочитано значений:", len(numbers))
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))


if __name__ == "__main__":
    main()
```

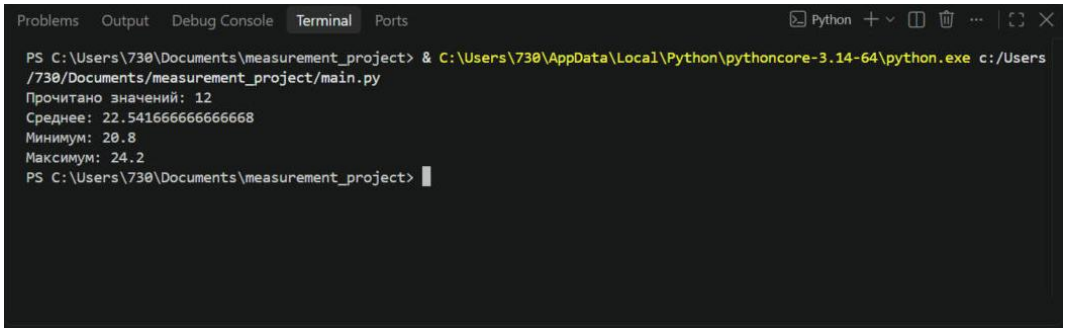


Рисунок 4.1. Чтение данных из файла

Задание 5. На рисунке 5: путь к входному файлу формируется с помощью Path("data") / "measurements.txt". Программа выводит текущий рабочий каталог, проверяет существование файла методом exists() (результат True) и

создаёт каталог logs вызовом mkdir(exist_ok=True), который не вызывает ошибки, если каталог уже существует.

```
from pathlib import Path

from utils.reader import read_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    print("Текущий рабочий каталог:", Path.cwd())

    input_path = Path("data") / "measurements.txt"
    print("Файл measurements.txt существует:",
input_path.exists())

    logs_dir = Path("logs")
    logs_dir.mkdir(exist_ok=True)

    numbers = read_numbers(input_path)
    print("Прочитано значений:", len(numbers))
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))


if __name__ == "__main__":
    main()
```

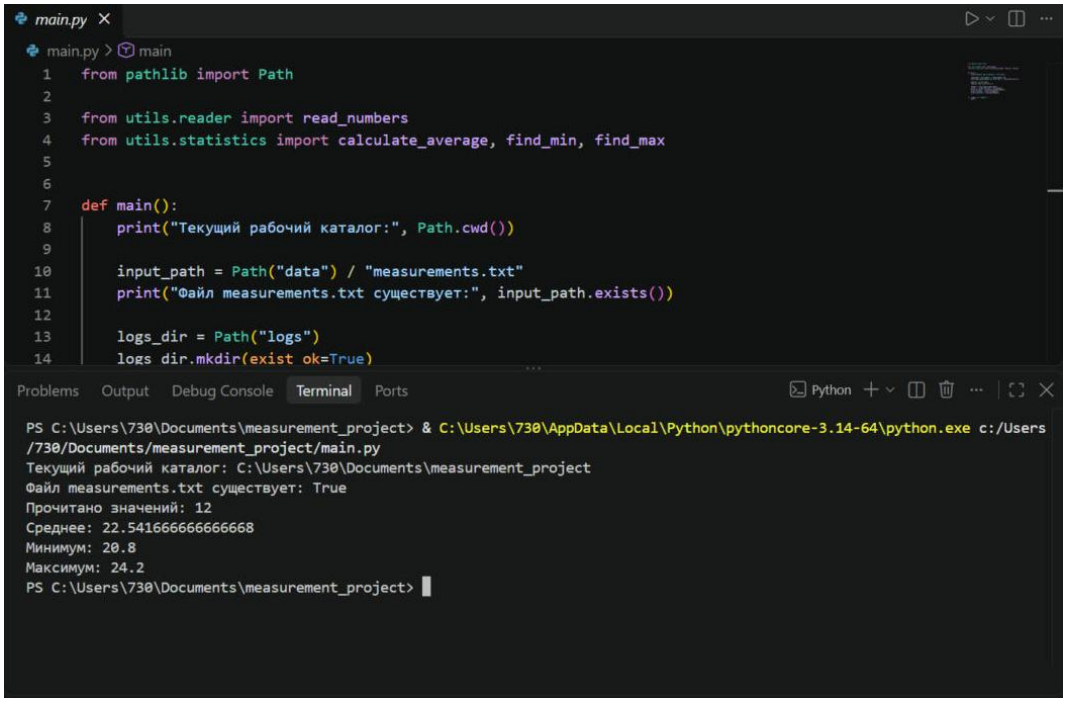


Рисунок 5. Работа с pathlib

Задание 6. На рисунке 6: при обращении к корректному файлу блок try выполняется без ошибок, выводятся статистические значения. На рисунке

6.1: путь изменён на отсутствующий файл data/missing.txt; исключение FileNotFoundError перехвачено, выведено сообщение «Ошибка: файл не найден». На рисунках 6.2 и 6.3: в пятую строку файла вместо числа записано abc; преобразование float("abc") вызывает ValueError, который перехвачен блоком except ValueError, программа вывела сообщение о некорректном значении и завершилась без аварийной остановки. После проверки в файле восстановлены исходные данные.

```
from pathlib import Path

from utils.reader import read_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    input_path = Path("data") / "measurements.txt"
    Path("logs").mkdir(exist_ok=True)

    try:
        numbers = read_numbers(input_path)
    except FileNotFoundError:
        print("Ошибка: файл не найден:", input_path)
        return
    except ValueError as error:
        print("Ошибка: в файле некорректное значение:", error)
        return

    print("Прочитано значений:", len(numbers))
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))


if __name__ == "__main__":
    main()
```

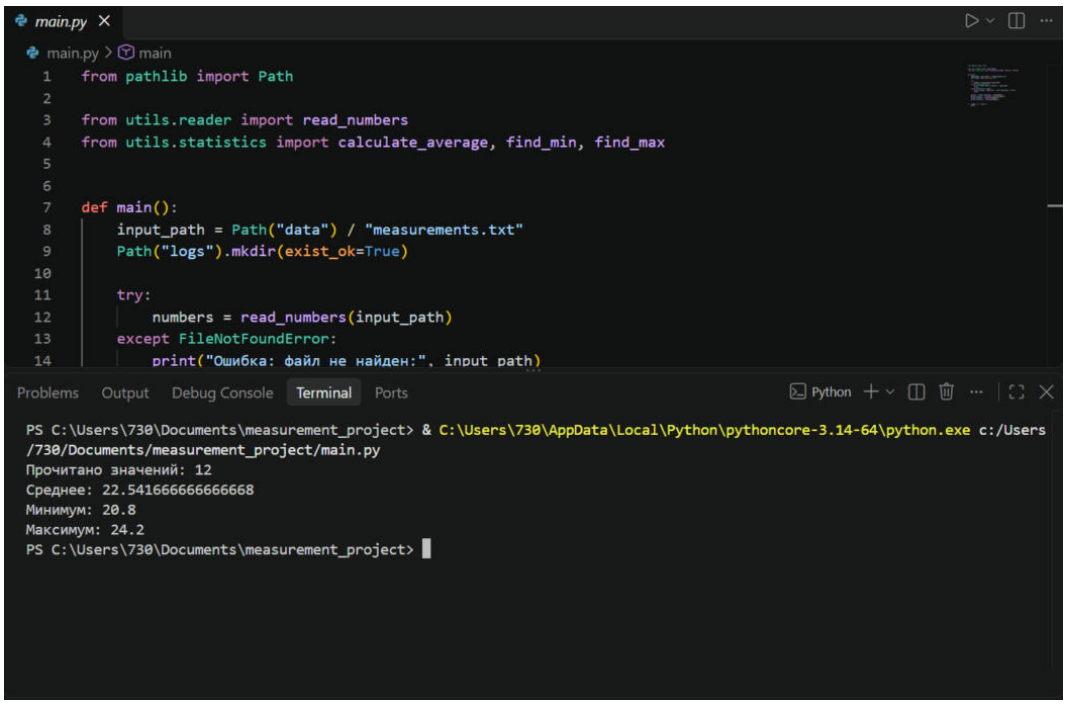


Рисунок 6. Работа программы с корректным файлом

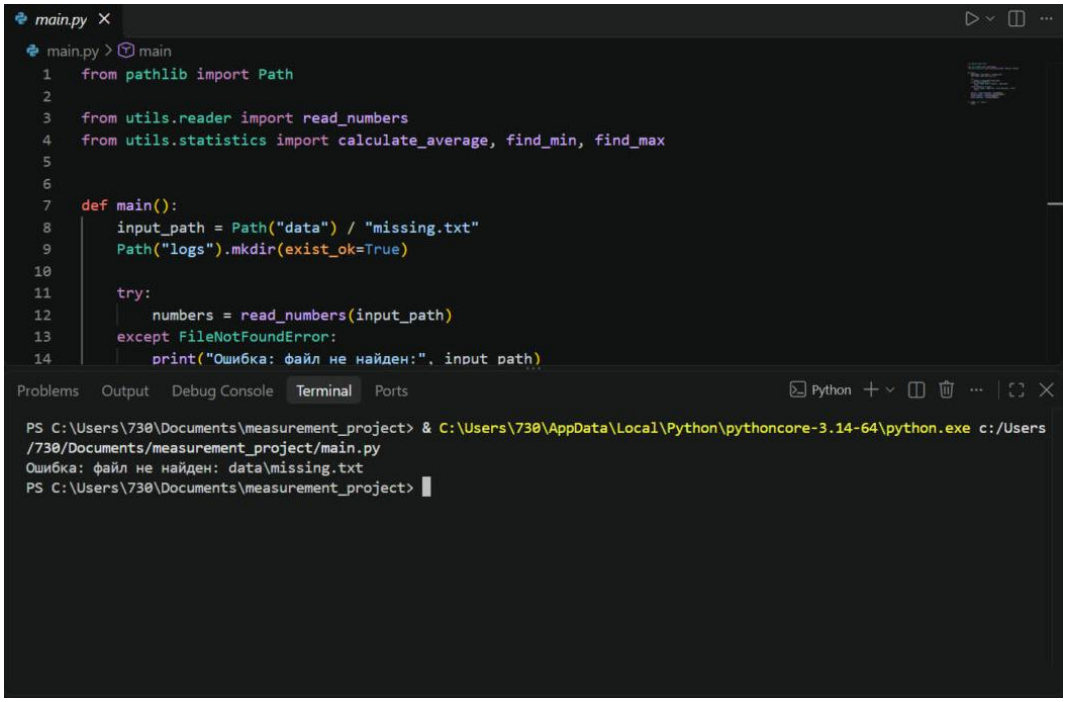


Рисунок 6.1. Обработка FileNotFoundError

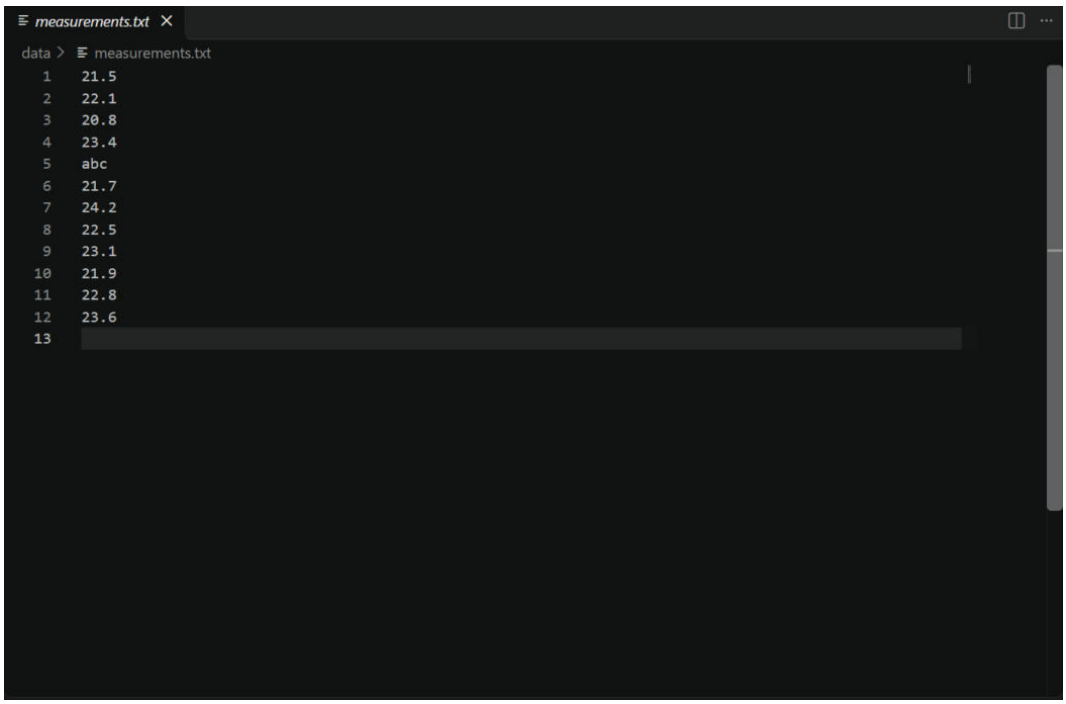


Рисунок 6.2. Некорректная строка в файле measurements.txt

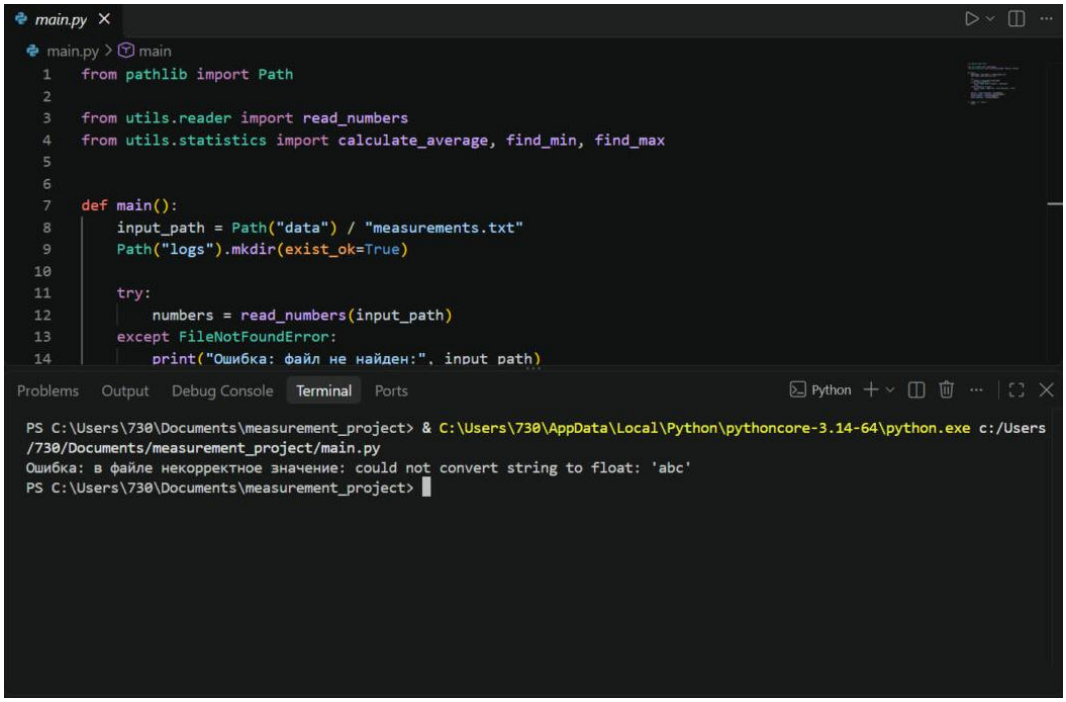


Рисунок 6.3. Обработка ValueError

Задание 7. На рисунке 7: в utils/reader.py создана функция validate_numbers(), которая при пустом списке выполняет raise ValueError("Список измерений пуст"). Функция вызывается в main.py сразу после чтения данных. Для проверки на время опыта использован пустой файл: исключение возникло и было обработано в основной логике программы, сообщение «Ошибка

данных: Список измерений пуст» выведено без трассировки. После проверки файл с данными восстановлен.

```
def read_numbers(file_path):
    numbers = []
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            line = line.strip()
            if line:
                numbers.append(float(line))
    return numbers


def validate_numbers(numbers):
    if len(numbers) == 0:
        raise ValueError("Список измерений пуст")
```

```
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    input_path = Path("data") / "measurements.txt"
    Path("logs").mkdir(exist_ok=True)

    try:
        numbers = read_numbers(input_path)
        validate_numbers(numbers)
    except FileNotFoundError:
        print("Ошибка: файл не найден:", input_path)
        return
    except ValueError as error:
        print("Ошибка данных:", error)
        return

    print("Прочитано значений:", len(numbers))
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))


if __name__ == "__main__":
    main()
```

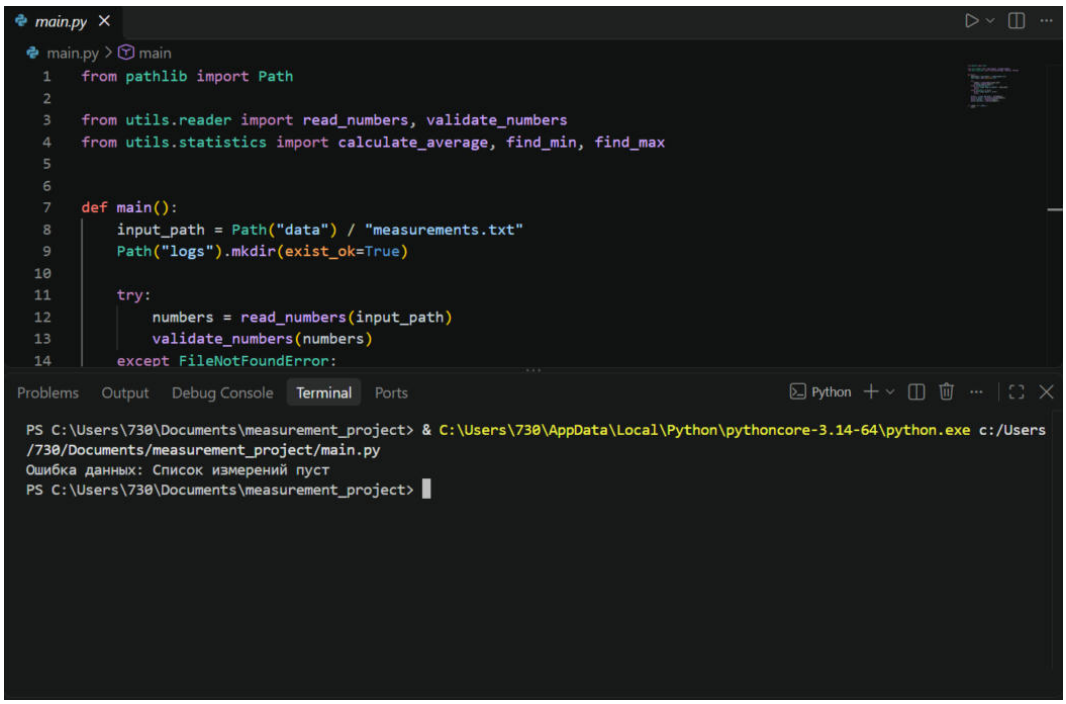


Рисунок 7. Проверка данных с помощью raise

Задание 8. На рисунке 8: подключён модуль logging, запись настроена в файл logs/app.log с форматом «дата и время, уровень, сообщение». В журнал записываются запуск программы, успешное чтение файла, количество прочитанных значений, ошибки и завершение программы. На рисунке 8.1:

содержимое app.log после запусков с корректным и отсутствующим файлами; ошибка записана с уровнем ERROR.

```
import logging
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max

Path("logs").mkdir(exist_ok=True)
logging.basicConfig(
    filename="logs/app.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    encoding="utf-8",
)


def main():
    logging.info("Программа запущена")
    input_path = Path("data") / "measurements.txt"

    try:
        numbers = read_numbers(input_path)
        validate_numbers(numbers)
    except FileNotFoundError:
        logging.error("Файл не найден: %s", input_path)
        print("Ошибка: файл не найден:", input_path)
        return
    except ValueError as error:
        logging.error("Ошибка данных: %s", error)
        print("Ошибка данных:", error)
        return

    logging.info("Файл %s успешно прочитан", input_path)
    logging.info("Прочитано значений: %d", len(numbers))
    print("Прочитано значений:", len(numbers))
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))
    logging.info("Программа завершена")


if __name__ == "__main__":
    main()
```

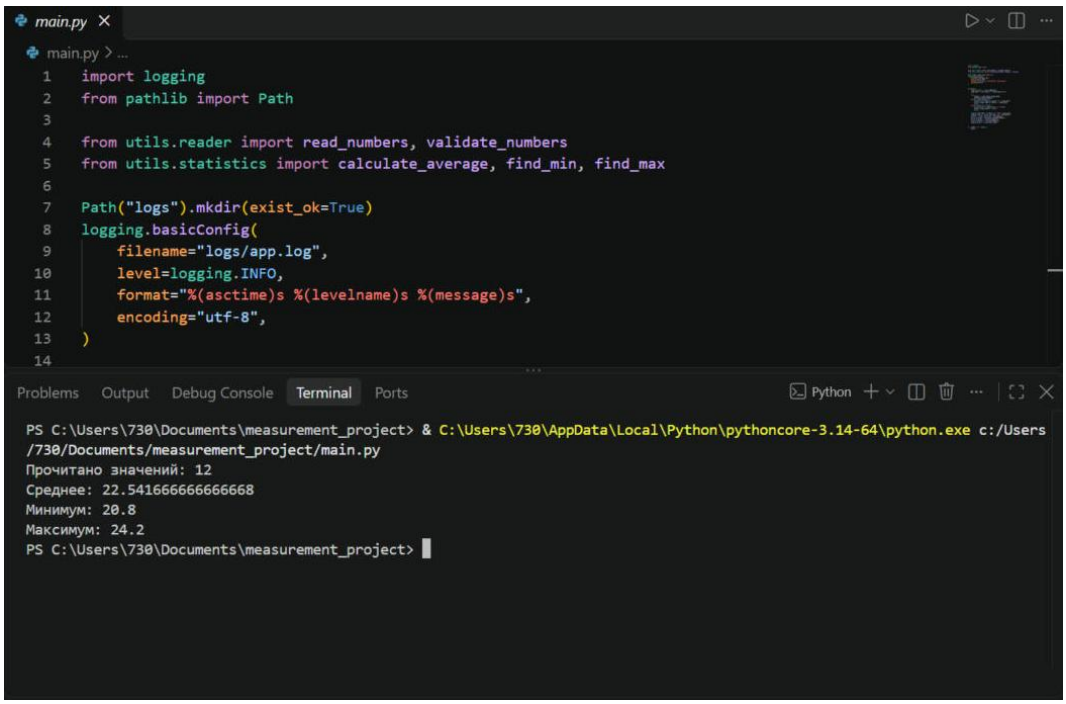


Рисунок 8. Настройка логирования

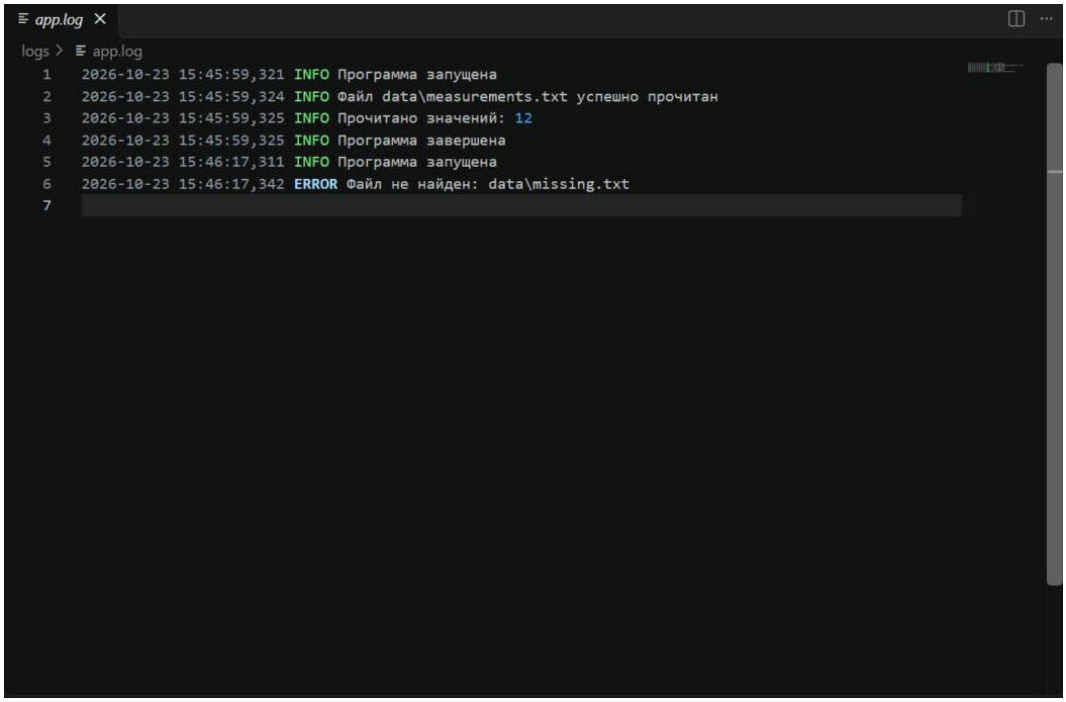


Рисунок 8.1. Содержимое файла app.log

Задание 9. На рисунке 9: создан config.json с параметрами input_file и log_file. В main.py добавлена функция load_config(), читающая настройки модулем json; путь к файлу измерений и путь к журналу берутся из конфигурации, а не из кода. На рисунке 9.1: запуск с исходным значением data/measurements.txt (12 значений). На рисунке 9.2: в config.json значение input_file изменено на data/measurements_2.txt; на рисунке 9.3 программа без

изменения исходного кода прочитала новый файл: 5 значений, среднее 10.6, минимум 9.5, максимум 12.0.

```
import json
import logging
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max


def load_config(config_path="config.json"):
    with open(config_path, "r", encoding="utf-8") as file:
        return json.load(file)


def main():
    config = load_config()

    Path("logs").mkdir(exist_ok=True)
    logging.basicConfig(
        filename=config["log_file"],
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
        encoding="utf-8",
    )

    logging.info("Программа запущена")
    input_path = Path(config["input_file"])
    print("Файл измерений из config.json:", input_path)

    try:
        numbers = read_numbers(input_path)
        validate_numbers(numbers)
    except FileNotFoundError:
        logging.error("Файл не найден: %s", input_path)
        print("Ошибка: файл не найден:", input_path)
        return
    except ValueError as error:
        logging.error("Ошибка данных: %s", error)
        print("Ошибка данных:", error)
        return

    logging.info("Файл %s успешно прочитан", input_path)
    logging.info("Прочитано значений: %d", len(numbers))
    print("Прочитано значений:", len(numbers))
    print("Среднее:", calculate_average(numbers))
    print("Минимум:", find_min(numbers))
    print("Максимум:", find_max(numbers))
    logging.info("Программа завершена")


if __name__ == "__main__":
    main()
```

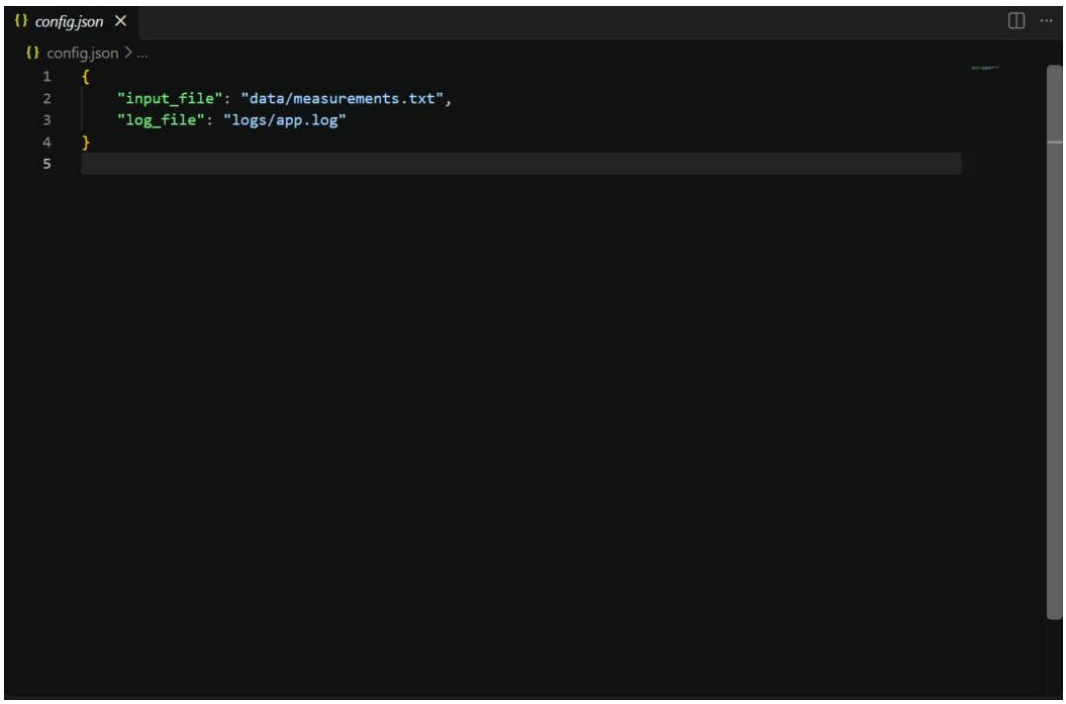


Рисунок 9. Файл config.json

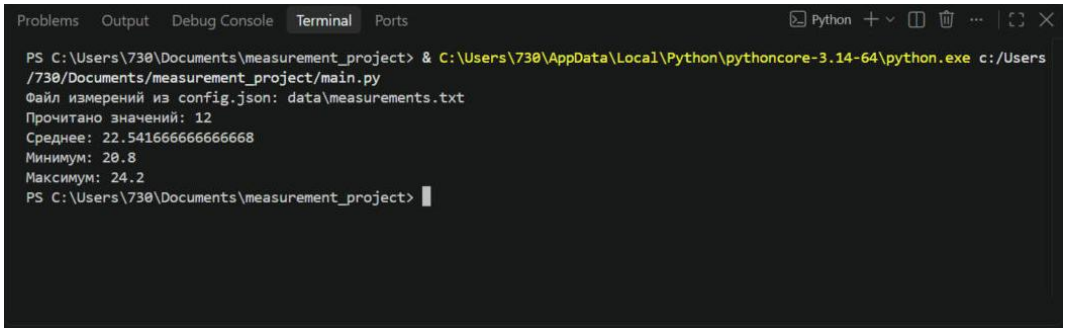


Рисунок 9.1. Запуск с исходной конфигурацией

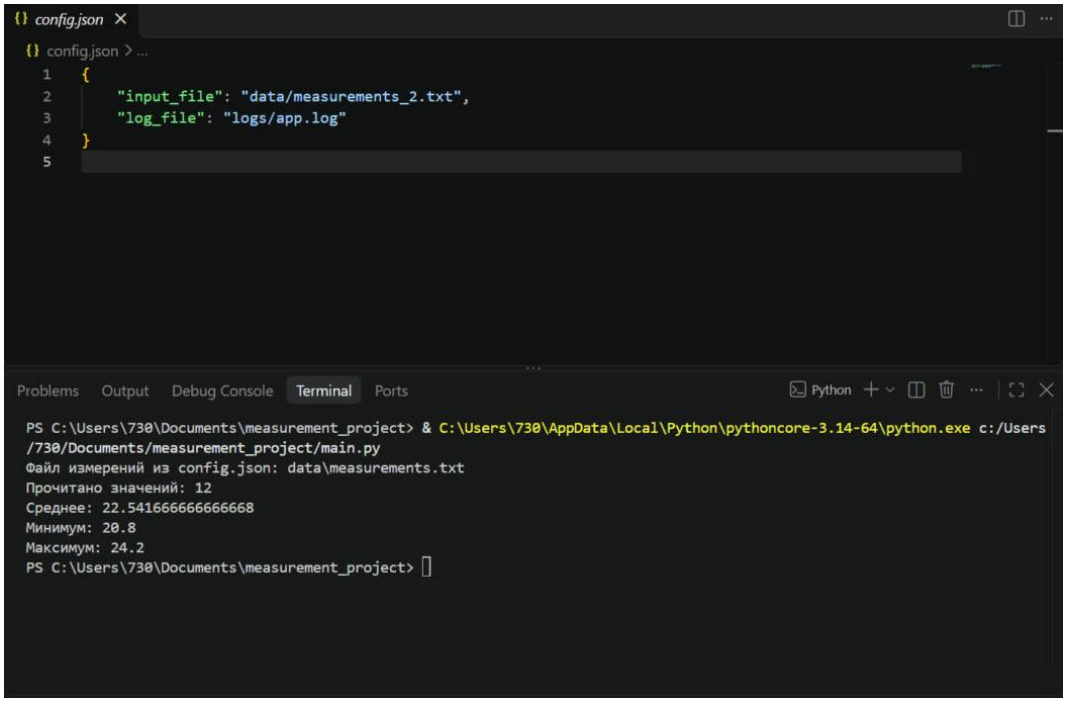


Рисунок 9.2. Изменённое значение input_file

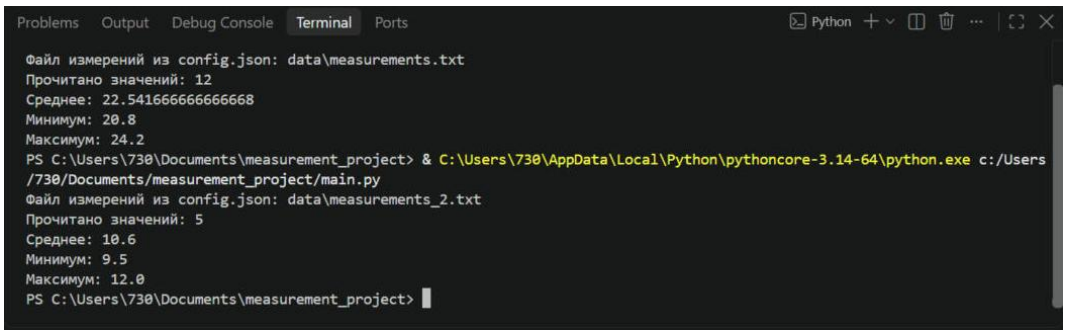


Рисунок 9.3. Запуск с новой конфигурацией

Задание 10. На рисунке 10: в корне проекта командой python -m venv .venv создано окружение (каталог .venv виден в проводнике). Первая попытка активации завершилась ошибкой PSSecurityException: PowerShell по умолчанию запрещает запуск скриптов. Ограничение снято только для текущего окна терминала командой Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, после чего команда .venv\Scripts\Activate.ps1 выполнена успешно, и в начале строки появилось (.venv). Команда python --version показала Python 3.14.7. В PowerShell where является псевдонимом другой команды и ничего не вывела, поэтому на рисунке 10.1 использована where.exe python: первым в списке указан C:\Users\730\Documents\measurement_project\.venv\Scripts\python.exe, то есть используется интерпретатор из каталога .venv.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python --version
where.exe python
```

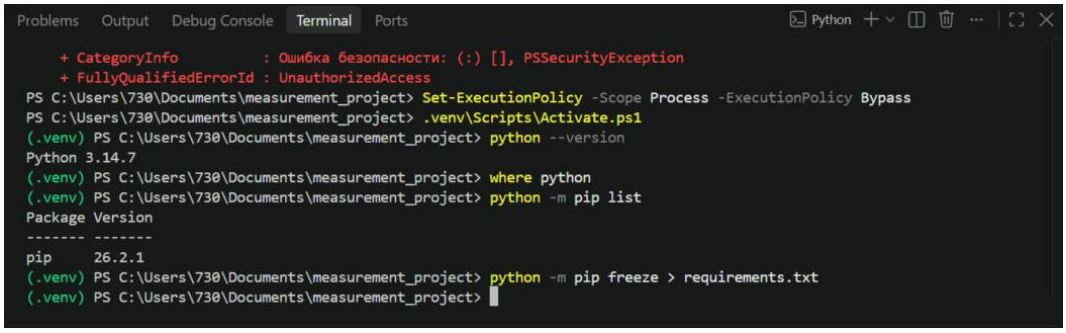


Рисунок 10. Активация окружения и версия Python

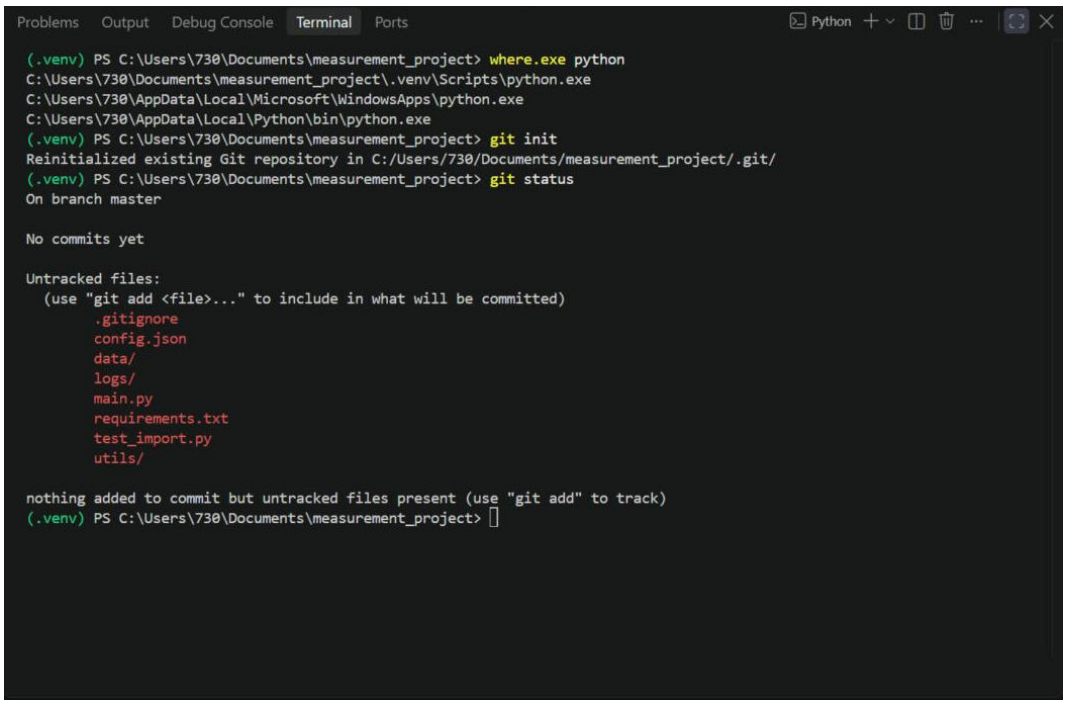


Рисунок 10.1. Путь к интерпретатору и git status

Задание 11. На рисунке 10: команда python -m pip list показала в окружении только пакет pip: сторонние библиотеки не устанавливались. Командой python -m pip freeze > requirements.txt создан файл requirements.txt, на рисунке 11 он открыт и пуст. Это корректный результат: pip freeze не включает сам pip, а проект использует только стандартную библиотеку Python (json, logging, pathlib), поэтому зависимостей для фиксации нет.

```
python -m pip list
python -m pip freeze > requirements.txt
```

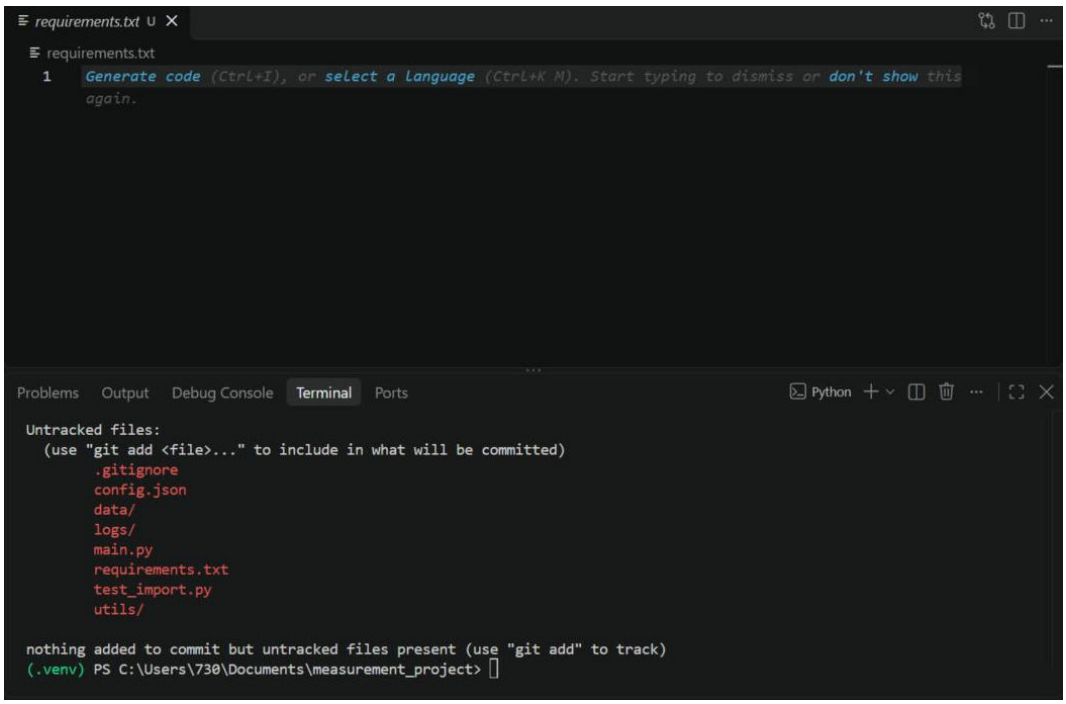


Рисунок 11. Файл requirements.txt

Задание 12. На рисунке 12: в .gitignore добавлены строки .venv/, __pycache__/, *.pyc и *.log. Репозиторий создан командой git init (на рисунке 10.1 при повторном вызове выведено «Reinitialized existing Git repository», то есть репозиторий уже существовал). Команда git status показывает в списке неотслеживаемых файлов .gitignore, config.json, data/, logs/, main.py, requirements.txt, test_import.py и utils/; каталоги .venv и __pycache__ в списке отсутствуют. Файл logs/app.log в Git также не предлагается: каталог logs/ указан из-за файла-заглушки .gitkeep, а app.log исключён правилом *.log.

```
git init
git status

.venv/
__pycache__/
*.pyc
*.log
```

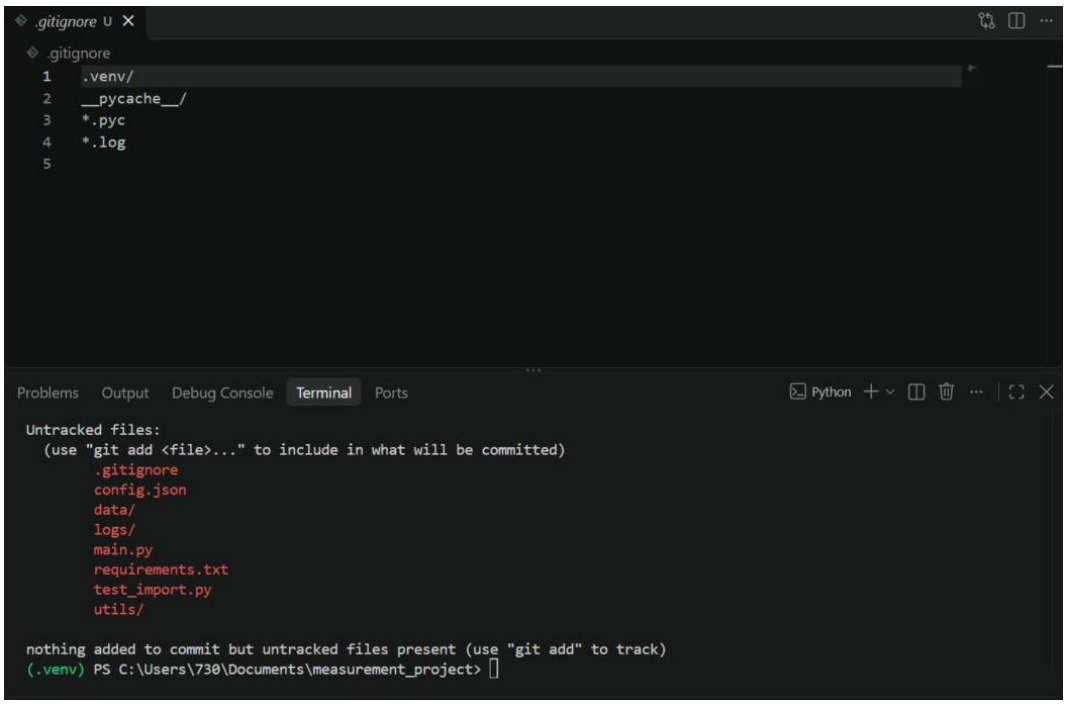


Рисунок 12. Файл .gitignore

Задание 13. Проект доведён до законченного состояния. Итоговая структура показана на рисунке 13: кроме обязательных файлов в ней остались учебные файлы предыдущих этапов (test_import.py, measurements_2.txt, measurements_bad.txt) и заглушки .gitkeep. Программа получает путь к данным из config.json, читает числа через utils.reader, проверяет данные функцией validate_numbers(), обрабатывает FileNotFoundError и ValueError, вычисляет количество значений, минимум, максимум и среднее функциями utils.statistics, пишет действия и ошибки в журнал и запускается через main() под условием if __name__ == "__main__". Запуск выполнялся интерпретатором из .venv (путь виден в строке запуска на рисунках 13.1–13.3). Проверены три сценария: корректный файл (рисунок 13.1: 12 значений, минимум 20.8, максимум 24.2, среднее 22.54), отсутствующий файл data/missing.txt (рисунок 13.2: «Ошибка: файл не найден») и файл с некорректным значением abc (рисунок 13.3: «Ошибка данных: could not

convert string to float: 'abc'»). Рисунок 13.4 показывает журнал app.log после запусков.

```
{
    "input_file": "data/measurements.txt",
    "log_file": "logs/app.log"
}
```

```
import json
import logging
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max


def load_config(config_path="config.json"):
    with open(config_path, "r", encoding="utf-8") as file:
        return json.load(file)


def setup_logging(log_file):
    Path(log_file).parent.mkdir(exist_ok=True)
    logging.basicConfig(
        filename=log_file,
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
        encoding="utf-8",
    )


def main():
    config = load_config()
    setup_logging(config["log_file"])
    logging.info("Программа запущена")

    input_path = Path(config["input_file"])
    print("Файл измерений:", input_path)

    try:
        numbers = read_numbers(input_path)
        validate_numbers(numbers)
    except FileNotFoundError:
        logging.error("Файл не найден: %s", input_path)
        print("Ошибка: файл не найден:", input_path)
        return
    except ValueError as error:
        logging.error("Ошибка данных: %s", error)
        print("Ошибка данных:", error)
        return

    logging.info("Файл %s успешно прочитан", input_path)
    logging.info("Прочитано значений: %d", len(numbers))

    count = len(numbers)
    minimum = find_min(numbers)
    maximum = find_max(numbers)
    average = calculate_average(numbers)

    print("Количество значений:", count)
    print("Минимум:", minimum)
    print("Максимум:", maximum)
    print(f"Среднее значение: {average:.2f}")
```

```
    logging.info("Количество: %d, минимум: %s, максимум: %s,
среднее: %.2f", count, minimum, maximum, average)
    logging.info("Программа завершена")


if __name__ == "__main__":
    main()
```

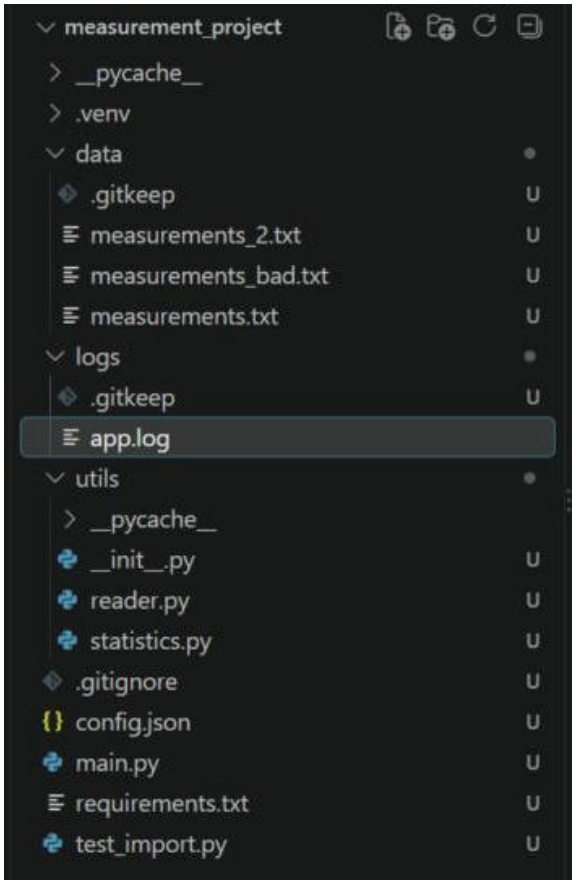


Рисунок 13. Итоговая структура проекта

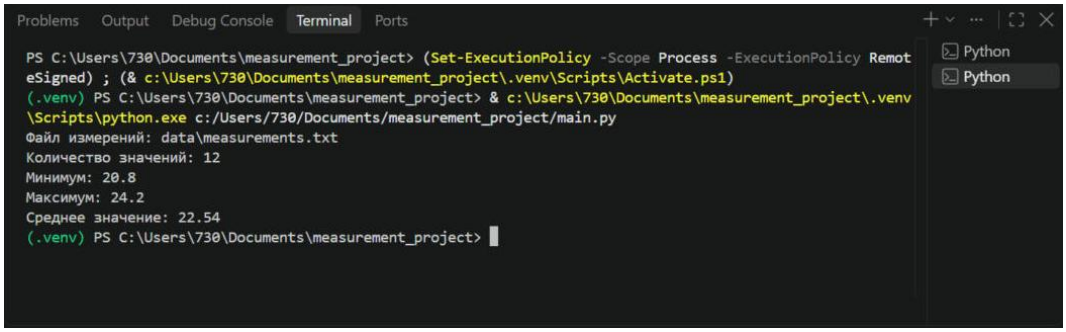


Рисунок 13.1. Сценарий 1: корректный файл

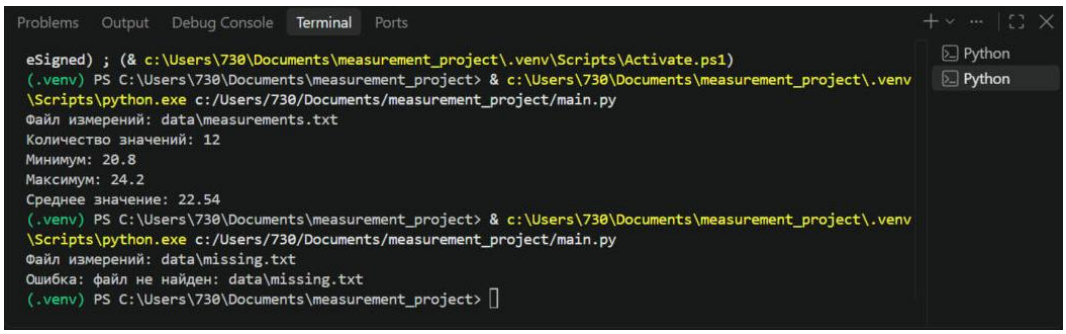


Рисунок 13.2. Сценарий 2: отсутствующий файл

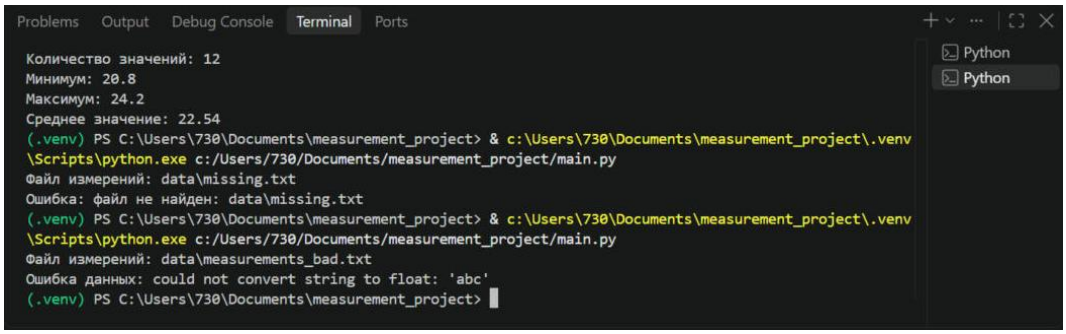


Рисунок 13.3. Сценарий 3: некорректное значение в файле

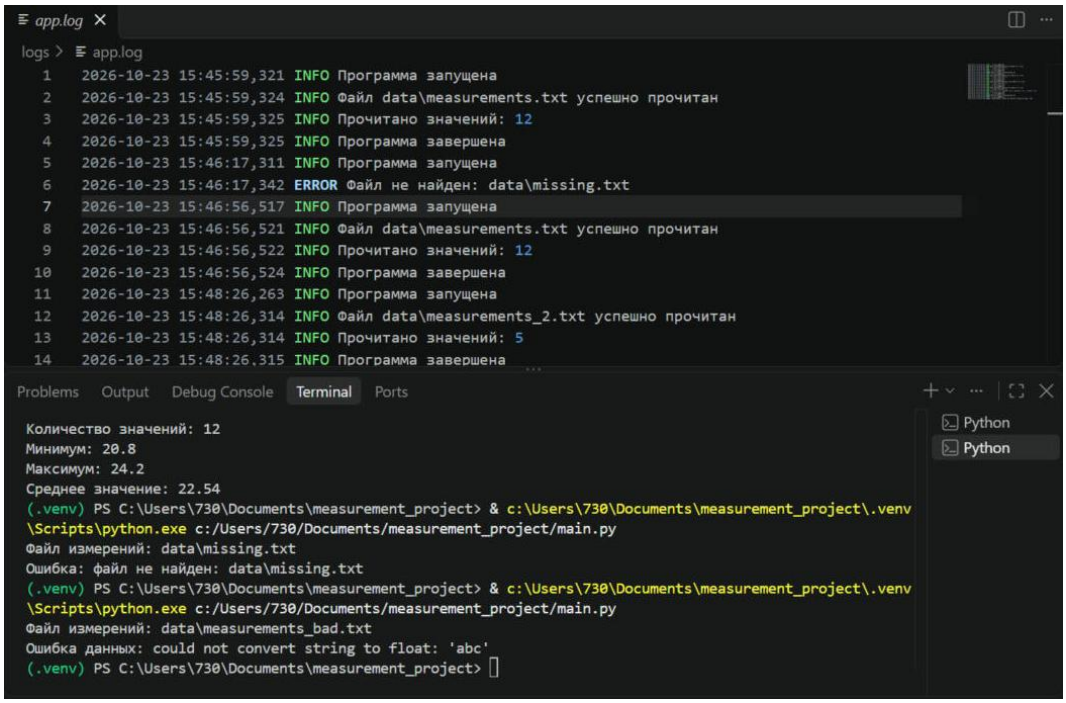


Рисунок 13.4. Журнал app.log

Вывод

В ходе лабораторной работы изучены принципы организации Python-проекта: программа разделена на точку входа main.py и пакет utils с модулями reader и statistics, входные данные вынесены в каталог data, журналы — в logs, настройки — в config.json. Использованы способы импорта from module import

..., а также конструкция if __name__ == "__main__", благодаря которой при импорте файла его функция main() не запускается.

Освоена работа с путями через pathlib (Path, exists(), mkdir(exist_ok=True)). Реализована обработка исключений FileNotFoundError и ValueError, а также самостоятельная проверка данных с помощью raise ValueError, что позволило программе сообщать об ошибках понятными сообщениями вместо аварийного завершения.

Применено логирование: события работы программы (запуск, чтение файла, число значений, ошибки, завершение) записываются в logs/app.log с датой, временем и уровнем сообщения. Параметры input_file и log_file вынесены в config.json, и замена входного файла выполнена без изменения исходного кода.

Создано виртуальное окружение .venv, проверено, что используется его интерпретатор, и сформирован пустой requirements.txt, что корректно для проекта на стандартной библиотеке. Настроен .gitignore (.venv/, __pycache__/, *.pyc, *.log), и по результату git status эти файлы в репозиторий не попадают. Итоговая программа проверена в трёх сценариях: корректный файл, отсутствующий файл и файл с некорректным значением.